



Chapter 3: Introduction to SQL

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Outline

- Overview of The SQL Query Language
- Table
 - SQL Data Definition
 - Queries
 - Modification of the Table



History

- IBM Sequel language developed as part of System R project at the IBM San Jose Research Laboratory, originally called **Sequel**.
- Renamed **Structured Query Language** (SQL)
- ANSI and ISO standard SQL:
 - SQL-86
 - SQL-89
 - SQL-92
 - SQL:1999 (language name became Y2K compliant!)
 - SQL:2003
- Commercial systems offer most, if not all, SQL-92 features, plus varying feature sets from later standards and special proprietary features.
 - Not all examples here may work on your particular system.



SQL的特点

- SQL的特点
 - 一体化
 - 功能一体化：提供数据定义、操纵和控制功能，集DDL、DML和DCL等语言于一体
 - 操作对象单一化：都是对关系进行操作
 - 高度非过程化
 - 用户只需提出“做什么”，而无须说明“怎么做”，不必介入具体的操作实现过程
 - 面向集合的操作方式
 - 一次一集合：每次操作的对象和结果都是关系 (记录集合)
 - 与之相反的是“一次一记录”：每次操作只能处理一条记录，要通过循环等手段才能处理一个记录集合



SQL Parts

- DML -- provides the ability to query information from the database and to insert tuples into, delete tuples from, and modify tuples in the database.
- integrity – the DDL includes commands for specifying integrity constraints.
- View definition -- The DDL includes commands for defining views.
- Transaction control –includes commands for specifying the beginning and ending of transactions.
- Embedded SQL and dynamic SQL -- define how SQL statements can be embedded within general-purpose programming languages.
- Authorization – includes commands for specifying access rights to relations and views.



Data Definition Language

The SQL data-definition language (DDL) allows the specification of information about relations, including:

- The schema for each relation.
- The type of values associated with each attribute.
- The Integrity constraints
- The set of indices to be maintained for each relation.
- Security and authorization information for each relation.
- The physical storage structure of each relation on disk.



SQL的数据定义功能

	创建	修改	删除
域	Create Domain	— —	Drop Domain
关系	Create Table	Alter Table	Drop Table
视图	Create View	— —	Drop View
索引	Create Index	— —	Drop Index
数据库	Create Database	— —	Drop Database



Domain Types in SQL

- **char(*n*)**. Fixed length character string, with user-specified length *n*.
- **varchar(*n*)**. Variable length character strings, with user-specified maximum length *n*.
- **int**. Integer (a finite subset of the integers that is machine-dependent).
- **smallint**. Small integer (a machine-dependent subset of the integer domain type).
- **numeric(*p*,*d*)**. Fixed point number, with user-specified precision of *p* digits, with *d* digits to the right of decimal point. (ex., **numeric**(3,1), allows 44.5 to be stored exactly, but not 444.5 or 0.32)
- **real, double precision**. Floating point and double-precision floating point numbers, with machine-dependent precision.
- **float(*n*)**. Floating point number, with user-specified precision of at least *n* digits.
- More are covered in Chapter 4.



Create Table Construct

- An SQL relation is defined using the **create table** command:

create table *r*

$(A_1 D_1, A_2 D_2, \dots, A_n D_n,$
(integrity-constraint₁),
...,
(integrity-constraint_k))

- *r* is the name of the relation
 - each A_i is an attribute name in the schema of relation *r*
 - D_i is the data type of values in the domain of attribute A_i
- Example:

create table *instructor* (
 ID **char**(5),
 name **varchar**(20),
 dept_name **varchar**(20),
 salary **numeric**(8,2))



Integrity Constraints in Create Table

- Types of integrity constraints
 - **primary key** ($A_1, \dots, A_n$)
 - **foreign key** ($A_m, \dots, A_n$) **references** r
 - **not null**
- SQL prevents any update to the database that violates an integrity constraint.
- Example:

```
create table instructor (  
    ID          char(5),  
    name        varchar(20) not null,  
    dept_name varchar(20),  
    salary      numeric(8,2),  
    primary key (ID),  
    foreign key (dept_name) references department);
```



And a Few More Relation Definitions

- **create table** *student* (
 ID **varchar**(5),
 name **varchar**(20) not null,
 dept_name **varchar**(20),
 tot_cred **numeric**(3,0),
 primary key (*ID*),
 foreign key (*dept_name*) **references** *department*);

- **create table** *takes* (
 ID **varchar**(5),
 course_id **varchar**(8),
 sec_id **varchar**(8),
 semester **varchar**(6),
 year **numeric**(4,0),
 grade **varchar**(2),
 primary key (*ID*, *course_id*, *sec_id*, *semester*, *year*) ,
 foreign key (*ID*) **references** *student*,
 foreign key (*course_id*, *sec_id*, *semester*, *year*) **references**
 section);



And more still

- **create table** *course* (
 course_id **varchar**(8),
 title **varchar**(50),
 dept_name **varchar**(20),
 credits **numeric**(2,0),
 primary key (*course_id*),
 foreign key (*dept_name*) **references** *department*);



实践：写出创建下面三个表的语句

- 例：

S

SC

C

<u>SNO</u>	Sname	AGE	SEX
------------	-------	-----	-----

<u>SNO</u>	<u>CNO</u>	RESULT
------------	------------	--------

<u>CNO</u>	Cname	Teacher
------------	-------	---------



Updates to tables

■ Drop Table

- **drop table r**
- 示例:
- **drop table SC**
- **Warning!** : 删除关系的定义后, 关系的结构和内容、相关索引、以及由它导出的视图都被删除。



Updates to tables

■ Alter

- **alter table r add A D**
 - where A is the name of the attribute to be added to relation r and D is the domain of A .
 - All existing tuples in the relation are assigned *null* as the value for the new attribute.
- **alter table r drop A**
 - where A is the name of an attribute of relation r
 - Dropping of attributes not supported by many databases.



SQL的数据查询功能

- 统一的查询语句：Select 语句
- 基本结构
 - Select 子句 (投影)
 - From 子句 (笛卡儿积)
 - Where 子句 (选择)
- 复合基本结构
 - 汇合多个基本结构查询出的结果 (集合运算)
- 扩展基本结构
 - 增加 Group By子句 (聚集运算)
 - 增加 Order By子句 (对结果排序)
 - 嵌套子查询
 - 连接关系



Basic Query Structure

- A typical SQL query has the form:

select $A_1, A_2, \dots, A_n$
from $r_1, r_2, \dots, r_m$
where P

- A_i represents an attribute
 - R_i represents a relation
 - P is a predicate.
- The result of an SQL query is a relation.
- 运算次序
 - From (笛卡儿积) $\rightarrow$ Where (选择) $\rightarrow$ Select (投影)



The select Clause

- The **select** clause lists the attributes desired in the result of a query
 - corresponds to the projection operation of the relational algebra
- Example: find the names of all instructors:

select *name*
from *instructor*
- NOTE: SQL names are case insensitive (i.e., you may use upper- or lower-case letters.)
 - E.g., *Name* $\equiv$ *NAME* $\equiv$ *name*
 - Some people use upper case wherever we use bold font.



The select Clause (Cont.)

- SQL allows duplicates in relations as well as in query results.
- To force the elimination of duplicates, insert the keyword **distinct** after select.
- Find the department names of all instructors, and remove duplicates

```
select distinct dept_name  
from instructor
```

- The keyword **all** specifies that duplicates should not be removed.

```
select all dept_name  
from instructor
```

-



The select Clause (Cont.)

- An asterisk in the select clause denotes “all attributes”

```
select *  
from instructor
```

- An attribute can be a literal with no **from** clause

```
select '437'
```

- Results is a table with one column and a single row with value “437”
- Can give the column a name using:

```
select '437' as FOO
```

- An attribute can be a literal with **from** clause

```
select 'A'  
from instructor
```

- Result is a table with one column and *N* rows (number of tuples in the *instructors* table), each row with value “A”



The select Clause (Cont.)

- The **select** clause can contain arithmetic expressions involving the operation, +, −, *, and /, and operating on constants or attributes of tuples.
 - The query:

```
select ID, name, salary/12  
from instructor
```

would return a relation that is the same as the *instructor* relation, except that the value of the attribute *salary* is divided by 12.

- Can rename “*salary/12*” using the **as** clause:

```
select ID, name, salary/12 as monthly_salary
```



The Rename Operation

- The SQL allows renaming relations and attributes using the **as** clause:

old-name as new-name

- Find the names of all instructors who have a higher salary than some instructor in 'Comp. Sci'.
 - **select distinct** *T.name*
from *instructor as T, instructor as S*
where *T.salary > S.salary and S.dept_name = 'Comp. Sci.'*
- Keyword **as** is optional and may be omitted
instructor as T $\equiv$ *instructor T*



Select 子句

- 作用：从(From子句列出的)关系中选出指定属性
 - 近似于投影运算
- 应用示例：
 - S： 学生关系
 - 问： 所有学生的姓名和年龄？

S

姓名	性别	年龄
小张	男	23
小李	女	22
小陈	男	22
小刘	女	22
小孙	男	22



姓名	年龄
小张	23
小李	22
小陈	22
小刘	22
小孙	22



Select 子句

- 要点
 - 属性列表中，可以用表达式来构造的新属性
 - 相当于广义投影运算
 - 例
 - S: 学生关系
 - 问: 每个学生的出生年份

姓名	性别	年龄
小张	男	23
小李	女	22
小陈	男	22
小刘	女	22
小孙	男	22



姓名	?
小张	2002
小李	2003
小陈	2003
小刘	2003
小孙	2003



Select 子句

- 用表达式构造的新属性无属性名，但可采用以下方式命名

表达式 **as** 属性名

- 原有属性也可用如下方式重新命名

旧属性名 **as** 新属性名

S

姓名	性别	年龄
小张	男	23
小李	女	22
小陈	男	22
小刘	女	22
小孙	男	22

?



学生姓名	出生年份
小张	2002
小李	2003
小陈	2003
小刘	2003
小孙	2003



Select 子句

■ 要点

- 如果列出的关系有多个，且有同名属性。则指定这些同名属性时，必须加关系名前缀：**关系名.属性名**。其它属性可加可不加
- 以下SQL正确么？如正确，结果关系的属性名是什么？

Select 姓名, 性别, 成绩

From R, S

S

姓名	性别	年龄
张军	男	23
王红	女	22
刘朝	男	22

Select R.姓名, S.姓名, S.性别

From R, S

R

姓名	课程	成绩
王红	物理	70
王红	数学	80
张军	数学	90
张军	物理	100
刘朝	数学	80



Select 子句

■ 要点

- 在属性列表中，*号表示所有关系的所有属性，关系名.*表示某个关系的所有属性
- 思考，下列SQL的结果包含哪些属性？

S

姓名	性别	年龄
张军	男	23
王红	女	22
刘朝	男	22

R

姓名	课程	成绩
王红	物理	70
王红	数学	80
张军	数学	90
张军	物理	100
刘朝	数学	80

Select * *From* S, R

S.姓名	课程	成绩	R.姓名	性别	年龄
.....					

Select S.*, R.成绩 *From* S, R

姓名	性别	年龄	成绩
.....			



思考

- select 子句等价于投影运算么？



Select 子句

■ 要点

- Select后面，属性列表前，可加上关键字all或者distinct
 - all关键字表明结果中不消去重复元组
 - distinct关键字表明结果中消去重复元组
 - 没说明all或distinct的情况下，默认是all，即不消去重复行

R

姓名	课程	成绩
王红	物理	70
王红	数学	80
张军	数学	90
张军	物理	100
刘朝	数学	80

Select all 课程
From R

课程
物理
数学
数学
物理
数学

Select distinct 课程
From R

课程
物理
数学



思考

- select 子句等价于投影运算么？
 - select distinct 等价于投影。select 或者 select all 都未消去重复



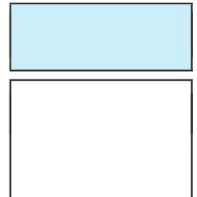
The where Clause

- The **where** clause specifies conditions that the result must satisfy
 - Corresponds to the selection predicate of the relational algebra.
- To find all instructors in Comp. Sci. dept

```
select name  
from instructor  
where dept_name = 'Comp. Sci.'
```

- SQL allows the use of the logical connectives **and**, **or**, and **not**
- The operands of the logical connectives can be expressions involving the comparison operators **<**, **<=**, **>**, **>=**, **=**, and **<>**.
- Comparisons can be applied to results of arithmetic expressions
- To find all instructors in Comp. Sci. dept with salary > 70000

```
select name  
from instructor  
where dept_name = 'Comp. Sci.' and salary > 70000
```





Where Clause Predicates

- SQL includes a **between** comparison operator
- Example: Find the names of all instructors with salary between \$90,000 and \$100,000 (that is, $\geq$ \$90,000 and $\leq$ \$100,000)
 - **select** *name*
from *instructor*
where *salary* **between** 90000 **and** 100000
- Tuple comparison
 - **select** *name, course_id*
from *instructor, teaches*
where (*instructor.ID, dept_name*) = (*teaches.ID*, 'Biology');



String Operations

- SQL includes a string-matching operator for comparisons on character strings. The operator **like** uses patterns that are described using two special characters:
 - percent (%). The % character matches any substring.
 - underscore (_). The _ character matches any character.
- Find the names of all instructors whose name includes the substring “dar”.

```
select name  
from instructor  
where name like '%dar%'
```

- Match the string “100%”

```
like '100 \%' escape '\'
```

in that above we use backslash (\) as the escape character.



String Operations (Cont.)

- **Patterns are case sensitive.**
- Pattern matching examples:
 - 'Intro%' matches any string beginning with “Intro”.
 - '%Comp%' matches any string containing “Comp” as a substring.
 - '___' matches any string of exactly three characters.
 - '___ %' matches any string of at least three characters.
- SQL supports a variety of string operations such as
 - concatenation (using “||”)
 - converting from upper to lower case (and vice versa)
 - finding string length, extracting substrings, etc.



Examples

- Find the names of all instructors who have taught some course and the course_id
 - **select** *name, course_id*
from *instructor , teaches*
where *instructor.ID = teaches.ID*
- Find the names of all instructors in the Art department who have taught some course and the course_id
 - **select** *name, course_id*
from *instructor , teaches*
where *instructor.ID = teaches.ID*
and *instructor. dept_name = 'Art'*

	-



Where 子句

- 作用：选出满足条件的行
 - 等价于选择运算
- 例：
 - S：学生关系
 - 问：所有年龄大于22岁的男学生的信息？

S

学号	姓名	性别	年龄
13	小张	男	23
02	小李	女	22
06	小陈	男	21
12	小刘	女	22
23	小孙	男	21

?



学号	姓名	性别	年龄
13	小张	男	23



Where 子句

- 在Where子句的条件中可能出现的运算符号
 - 比较运算符
 - $<$ 、 $\leq$ 、 $>$ 、 $\geq$ 、 $=$ 、 $<>$
 - 逻辑运算符
 - and, or, not
 - 范围运算
 - 格式1: between 下界 and 上界 (下界 $<$ 上界)
 - 格式2: not between 下界 and 上界
 - 例1: 年龄 between 20 and 23
 - 例2: 年龄 between 23 and 20



Where 子句

- 集合运算:
 - 格式1: `in` 集合;
 - 格式2: `not in` 集合
 - 例: 姓名 `in` { '小陈' , '小李' }
 - 例: 姓名 `not in` { '小陈' , '小李' }
- 匹配运算
 - 格式: `like` 匹配串; `not like` 匹配串
 - \是转义字符
 - 例1: 姓名 `like` '陈%', 匹配陈毅、陈小鲁, 0-n个字符
 - 例2: 姓名 `like` '陈_', 只能匹配陈毅, 1个字符
 - 例3: 内容 `like` '%100\%正确%' 匹配含'100%正确'的字符串



Where 子句

- 运算顺序：例：

- S：学生关系
- 问：所有学生的姓名和出生年份？

S

Select 姓名, 2025 - 年龄 *as* 出生年份
From S

学号	姓名	性别	年龄
13	小张	男	23
02	小李	女	22
06	小陈	男	21
12	小刘	女	22
23	小孙	男	21



姓名	出生年份
小张	2002
小李	2003
小陈	2004
小刘	2003
小孙	2004



Where 子句

■ 例:

- S : 学生关系
- 问: 所有99年之前(不含99年)出生的学生姓名和出生年份?

S

学号	姓名	性别	年龄
13	小张	男	23
02	小李	女	22
06	小陈	男	21
12	小刘	女	22
23	小孙	男	21

Select 姓名, 2025 - 年龄 **as** 出生年份

From S

Where 出生年份 < 1999



?



思考

- `select` 子句中，可以为属性更名。那么 `Where` 子句中，应使用新属性名还是旧属性名？



思考

- **select** 子句中，可以为属性更名。那么Where子句中，应使用新属性名还是旧属性名？
 - 因为选择运算(Where子句)在投影运算(Select子句)之前。所以Where子句中不能使用Select子句中更名后的新属性名



The from Clause

- The **from** clause lists the relations involved in the query
 - Corresponds to the Cartesian product operation of the relational algebra.
- Find the Cartesian product *instructor X teaches*

```
select *  
from instructor, teaches
```

- generates every possible instructor – teaches pair, with all attributes from both relations.
 - For common attributes (e.g., *ID*), the attributes in the resulting table are renamed using the relation name (e.g., *instructor.ID*)
- Cartesian product not very useful directly, but useful combined with where-clause condition (selection operation in relational algebra).



From 子句

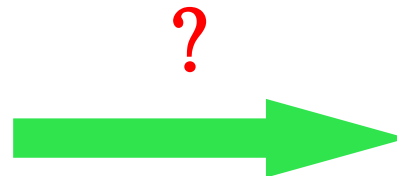
- 作用：连接多个关系的元组
 - 等价于笛卡儿积运算
 - 注：连接元组时如果有条件，这些条件应该加到 Where 子句中
- 例：
 - S：学生表； C： 班级表；
 - 问： `select * from S, C` 的结果是？
 - 问： 所有学生的姓名和班主任？

S

姓名	班号
小张	1
小刘	2
小李	1

C

班号	班主任
1	老王
2	老陈



姓名	班主任
小张	老王
小刘	老陈
小李	老王



From 子句

- 要点
 - From子句中，关系可用如下方式重命名
旧关系名 as 新关系名
 - 等价于更名运算
 - 一个关系在From子句中多次出现时，从第二次开始必须重命名
 - 关系重命名后，Select和Where子句中的前缀是使用新的关系名



From 子句

- 例:
 - 关系R: 选修成绩
 - 问: 谁的物理成绩高于王红?

R

姓名	课程	成绩
王红	物理	70
王红	数学	80
张军	数学	90
张军	物理	100
刘朝	物理	60

R as S

姓名	课程	成绩
王红	物理	70
王红	数学	80
张军	数学	90
张军	物理	100
刘朝	数学	60

?



Nested Subqueries

- SQL provides a mechanism for the nesting of subqueries. A **subquery** is a **select-from-where** expression that is nested within another query.
- The nesting can be done in the following SQL query

select $A_1, A_2, \dots, A_n$
from $r_1, r_2, \dots, r_m$
where P

as follows: (运算次序: From (笛卡儿积) $\rightarrow$ [Where (选择)] $\rightarrow$ [Select (投影)])

- **From clause:** r_i can be replaced by any valid subquery
- **Where clause:** P can be replaced with an expression of the form:

$B <\text{operation}> (\text{subquery})$

B is an attribute and $<\text{operation}>$ to be defined later.

- **Select clause:**

A_i can be replaced by a subquery that generates a single value.



Set Operations

- Find courses that ran in Fall 2017 or in Spring 2018
(**select** *course_id* **from** *section* **where** *sem* = 'Fall' **and** *year* = 2017)
union
(**select** *course_id* **from** *section* **where** *sem* = 'Spring' **and** *year* = 2018)
- Find courses that ran in Fall 2017 and in Spring 2018
(**select** *course_id* **from** *section* **where** *sem* = 'Fall' **and** *year* = 2017)
intersect
(**select** *course_id* **from** *section* **where** *sem* = 'Spring' **and** *year* = 2018)
- Find courses that ran in Fall 2017 but not in Spring 2018
(**select** *course_id* **from** *section* **where** *sem* = 'Fall' **and** *year* = 2017)
except
(**select** *course_id* **from** *section* **where** *sem* = 'Spring' **and** *year* = 2018)



Set Operations (Cont.)

- Set operations **union**, **intersect**, and **except**
 - Each of the above operations automatically eliminates duplicates
- To retain all duplicates use the
 - **union all**,
 - **intersect all**
 - **except all**.



集合运算

■ 例

- R: 选修关系
- 同时选修了物理和数学的学生姓名？

R

姓名	课程	成绩
王红	物理	70
王红	数学	80
张军	数学	90
张军	化学	100
刘朝	物理	80

Select 姓名

From R

Where 课程=“物理”
and 课程=“数学”

姓名



?

姓名

王红



集合运算

- 例：选修化学或者数学的学生姓名？

R

姓名	课程	成绩
王红	物理	70
王红	数学	80
张军	数学	90
张军	化学	100
刘朝	物理	80

Select distinct 姓名

From R

Where 课程=“化学”

or 课程=“数学”

?

姓名
王红
张军



姓名
王红
张军



集合运算

- 例：没选修数学的学生姓名？

R

姓名	课程	成绩
王红	物理	70
王红	数学	80
张军	数学	90
张军	化学	100
刘朝	物理	80

Select 姓名

From R

Where 课程<>“数学”

?

姓名
王红
张军
刘朝



姓名
刘朝



Null Values

- It is possible for tuples to have a null value, denoted by **null**, for some of their attributes
- **null** signifies an unknown value or that a value does not exist.
- The result of any arithmetic expression involving **null** is **null**
 - Example: $5 + \text{null}$ returns **null**
- The predicate **is null** can be used to check for null values.
 - Example: Find all instructors whose salary is null.

```
select name  
from instructor  
where salary is null
```

- The predicate **is not null** succeeds if the value on which it is applied is not null.



Null Values (Cont.)

- SQL treats as **unknown** the result of any comparison involving a null value (other than predicates **is null** and **is not null**).
 - Example: $5 < \text{null}$ or $\text{null} \diamond \text{null}$ or $\text{null} = \text{null}$
- The predicate in a **where** clause can involve Boolean operations (**and**, **or**, **not**); thus the definitions of the Boolean operations need to be extended to deal with the value **unknown**.
 - **and** : $(\text{true and unknown}) = \text{unknown}$,
 $(\text{false and unknown}) = \text{false}$,
 $(\text{unknown and unknown}) = \text{unknown}$
 - **or**: $(\text{unknown or true}) = \text{true}$,
 $(\text{unknown or false}) = \text{unknown}$
 $(\text{unknown or unknown}) = \text{unknown}$
- Result of **where** clause predicate is treated as *false* if it evaluates to *unknown*



Aggregate Functions

- These functions operate on the multiset of values of a column of a relation, and return a value

avg: average value

min: minimum value

max: maximum value

sum: sum of values

count: number of values



Aggregate Functions Examples

- Find the average salary of instructors in the Computer Science department
 - **select avg** (*salary*)
from *instructor*
where *dept_name*= 'Comp. Sci.';
- Find the total number of instructors who teach a course in the Spring 2018 semester
 - **select count** (**distinct** *ID*)
from *teaches*
where *semester* = 'Spring' **and** *year* = 2018;
- Find the number of tuples in the *course* relation
 - **select count** (*)
from *course*;



Aggregate Functions – Group By

- Find the average salary of instructors in each department
 - **select** *dept_name*, **avg** (*salary*) **as** *avg_salary*
from *instructor*
group by *dept_name*;

		-	

-	-



Aggregation (Cont.)

- Attributes in **select** clause outside of aggregate functions must appear in **group by** list
 - /* erroneous query */
select *dept_name*, *ID*, **avg** (*salary*)
from *instructor*
group by *dept_name*;



Aggregate Functions – Having Clause

- Find the names and average salaries of all departments whose average salary is greater than 42000

```
select dept_name, avg (salary) as avg_salary  
from instructor  
group by dept_name  
having avg (salary) > 42000;
```

- Note: predicates in the **having** clause are applied after the formation of groups whereas predicates in the **where** clause are applied before forming groups



Group By 子句

- 例：
 - 关系R: 选修成绩
 - 问: 所有课程的平均成绩和最高成绩?

R

姓名	课程	成绩
王红	物理	70
王红	数学	80
张军	数学	90
张军	物理	100
刘朝	物理	80

?

平均成绩	最高成绩
84	100



Group By 子句

- 例：
 - 关系R: 选修成绩
 - 问: 每门课程的平均和最高成绩?

R

姓名	课程	成绩
王红	物理	70
王红	数学	80
张军	数学	90
张军	物理	100
刘朝	物理	80

?

课程	平均成绩	最高成绩
物理	83.3	100
数学	85	95



Having 子句

- - Select* $A_1, A_2, \dots, A_n$
 - From* $R_1, R_2, \dots, R_m$
 - Where* P (元组限定条件)
 - Group By* 属性1, 属性2, ...
 - Having* Q (分组限定条件)
- **Having子句只能配合Group By子句使用，而不能单独出现**
- Having子句作用：在分组后，筛选满足条件Q的分组
 - 在分组限定条件中出现的属性只能是以下形式：
 - ① 分组属性
 - ② 聚集函数(任意属性)



Having 子句

- Having子句中的条件和Where子句中条件的不同
 - Where子句中的条件用于限定元组，施加在单个元组上
 - 不满足条件的元组将不会参与到下一步的分组运算(Group By子句)或投影运算(Select 子句)等
 - Having子句中的条件用于限定Group By子句产生的各个分组，施加在整个分组上
 - 不满足条件的分组，将不会参与下一步的统计运算(Select子句)



Having 子句

- 例:
 - 关系R: 选修成绩
 - 问: 平均成绩大于85的学生姓名?

R

姓名	课程	成绩
王红	物理	70
王红	数学	90
张军	数学	90
张军	物理	100
刘朝	物理	80

?

姓名
张军



Having 子句

- 问: 成绩大于85的学生姓名? (对比以上问题)

R

姓名	课程	成绩
王红	物理	70
王红	数学	90
张军	数学	90
张军	物理	100
刘朝	物理	80

Select distinct 姓名

From R

Where 成绩 > 85

姓名
王红
张军



Ordering the Display of Tuples

- List in alphabetic order the names of all instructors

```
select distinct name  
from instructor  
order by name
```

- We may specify **desc** for descending order or **asc** for ascending order, for each attribute; ascending order is the default.
 - Example: **order by** *name* **desc**
- Can sort on multiple attributes
 - Example: **order by** *dept_name*, *name*



Order By 子句

- - Select* $A_1, A_2, \dots, A_n$
 - From* $R_1, R_2, \dots, R_m$
 - [Where P]*
 - [Group By* 属性1, 属性2, ...
 - [Having* 分组限定条件Q *]*
 - [Order By* 属性1 [*asc* | *desc*],
 - 属性2 [*asc* | *desc*], ... *]*
- Order By子句的作用：在Select子句得出结果后，对结果进行排序
 - 先按属性1的值，升序(*asc*)或降序(*desc*)排列，缺省是升序；属性1的值相同时，再按属性2值升序或降序排列...



Order By 子句

- 例
 - R: 选修关系
 - 问: 从高到低列出物理课程的成绩

R

姓名	课程	成绩
王红	物理	70
王红	数学	80
张军	数学	90
张军	物理	100
刘朝	数学	80

?

姓名	成绩
张军	100
王红	70



Select 语句的运算次序

- 较完整结构

- - Select* $A_1, A_2, \dots, A_n$
From $R_1, R_2, \dots, R_m$
[Where 元组限定条件 P]
[Group By 属性1, 属性2, ...
[Having 分组限定条件 Q]]
*[Order By 属性1 [*asc* | *desc*],*
*属性2 [*asc* | *desc*], ...]*

- 运算次序

- From (笛卡儿积) $\rightarrow$ [Where (选择)] $\rightarrow$ [Group By (分组)]
 $\rightarrow$ [Having (限定分组)] $\rightarrow$ [Select (投影, 或统计)] $\rightarrow$
[Order By (结果排序)]



Set Membership



Set Membership

- Find courses offered in Fall 2017 and in Spring 2018

```
select distinct course_id  
from section  
where semester = 'Fall' and year= 2017 and  
       course_id in (select course_id  
                      from section  
                      where semester = 'Spring' and year= 2018);
```

- Find courses offered in Fall 2017 but not in Spring 2018

```
select distinct course_id  
from section  
where semester = 'Fall' and year= 2017 and  
       course_id not in (select course_id  
                          from section  
                          where semester = 'Spring' and year= 2018);
```



Set Membership (Cont.)

- Name all instructors whose name is neither “Mozart” nor Einstein”

```
select distinct name  
from instructor  
where name not in ('Mozart', 'Einstein')
```

- Find the total number of (distinct) students who have taken course sections taught by the instructor with *ID* 10101

```
select count (distinct ID)  
from takes  
where (course_id, sec_id, semester, year) in  
      (select course_id, sec_id, semester, year  
       from teaches  
       where teaches.ID= 10101);
```

- Note: Above query can be written in a much simpler manner. The formulation above is simply to illustrate SQL features



在Where子句中 —— 用子查询构造条件

■ 例

- R: 选修关系
- 问: 没有选修数学的学生姓名?

R

姓名	课程	成绩
王红	物理	70
王红	数学	50
张军	数学	90
张军	物理	100
刘朝	物理	40

?

姓名
刘朝



Set Comparison



Set Comparison – “some” Clause

- Find names of instructors with salary greater than that of some (at least one) instructor in the Biology department.

```
select distinct T.name  
from instructor as T, instructor as S  
where T.salary > S.salary and S.dept name = 'Biology';
```

- Same query using > **some** clause

```
select name  
from instructor  
where salary > some (select salary  
                        from instructor  
                        where dept name = 'Biology');
```



Definition of “some” Clause

- $F \text{ <comp> some } r \Leftrightarrow \exists t \in r \text{ such that } (F \text{ <comp> } t)$
Where <comp> can be: $<, \leq, >, =, \neq$

$$(5 < \text{some } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline 6 \\ \hline \end{array}) = \text{true}$$

(read: 5 < some tuple in the relation)

$$(5 < \text{some } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline \end{array}) = \text{false}$$

$$(5 = \text{some } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline \end{array}) = \text{true}$$

$$(5 \neq \text{some } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline \end{array}) = \text{true (since } 0 \neq 5)$$

$(= \text{some}) \equiv \text{in}$

However, $(\neq \text{some}) \not\equiv \text{not in}$



Set Comparison – “all” Clause

- Find the names of all instructors whose salary is greater than the salary of all instructors in the Biology department.

```
select name  
from instructor  
where salary > all (select salary  
                        from instructor  
                        where dept name = 'Biology');
```



Definition of “all” Clause

- $F \text{ <comp> all } r \Leftrightarrow \forall t \in r (F \text{ <comp> } t)$

$$(5 < \text{all } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline 6 \\ \hline \end{array}) = \text{false}$$

$$(5 < \text{all } \begin{array}{|c|} \hline 6 \\ \hline 10 \\ \hline \end{array}) = \text{true}$$

$$(5 = \text{all } \begin{array}{|c|} \hline 4 \\ \hline 5 \\ \hline \end{array}) = \text{false}$$

$$(5 \neq \text{all } \begin{array}{|c|} \hline 4 \\ \hline 6 \\ \hline \end{array}) = \text{true (since } 5 \neq 4 \text{ and } 5 \neq 6)$$

$(\neq \text{all}) \equiv \text{not in}$

However, $(= \text{all}) \not\equiv \text{in}$



■ 例

- R: 选修关系
- 问: 物理课哪个学生的成绩最高?

R

姓名	课程	成绩
王红	物理	70
王红	数学	50
张军	数学	90
张军	物理	100
刘朝	物理	40

?

姓名	最高物理成绩
张军	100



Test for Empty Relations

- The **exists** construct returns the value **true** if the argument subquery is nonempty.
- **exists** $r \Leftrightarrow r \neq \emptyset$
- **not exists** $r \Leftrightarrow r = \emptyset$



Use of “exists” Clause

- Yet another way of specifying the query “Find all courses taught in both the Fall 2017 semester and in the Spring 2018 semester”

```
select course_id
from section as S
where semester = 'Fall' and year = 2017 and
      exists (select *
              from section as T
              where semester = 'Spring' and year = 2018
                  and S.course_id = T.course_id);
```

- **Correlation name** – variable *S* in the outer query
- **Correlated subquery** – the inner query



Use of “not exists” Clause

- Find all students who have taken all courses offered in the Biology department.

```
select distinct S.ID, S.name  
from student as S  
where not exists ( (select course_id  
                    from course  
                    where dept_name = 'Biology')  
except  
                (select T.course_id  
                 from takes as T  
                 where S.ID = T.ID));
```

- First nested query lists all courses offered in Biology
 - Second nested query lists all courses a particular student took
- Note that $X - Y = \emptyset \Leftrightarrow X \subseteq Y$
- Note: Cannot write this query using = all and its variants



Test for Absence of Duplicate Tuples

- The **unique** construct tests whether a subquery has any duplicate tuples in its result.
- The **unique** construct evaluates to “true” if a given subquery contains no duplicates .
- Find all courses that were offered at most once in 2017

```
select T.course_id
from course as T
where unique ( select R.course_id
                  from section as R
                  where T.course_id= R.course_id
                      and R.year = 2017);
```



在Where子句中 —— 用子查询构造条件

■ 例

- R: 选修关系; S: 学生关系
- 问: 哪个学生没有选修任何课程? 列出姓名和年龄

S

姓名	性别	年龄
王红	女	23
张军	男	22
刘朝	男	22
李丽	女	24

R

姓名	课程	成绩
王红	物理	70
王红	数学	80
张军	数学	90
张军	物理	100
刘朝	数学	80

?

姓名	年龄
李丽	24



Subqueries in the From Clause

- SQL allows a subquery expression to be used in the **from** clause
- Find the average instructors' salaries of those departments where the average salary is greater than \$42,000."

```
select dept_name, avg_salary
from ( select dept_name, avg (salary) as avg_salary
      from instructor
      group by dept_name)
where avg_salary > 42000;
```

- Note that we do not need to use the **having** clause
- Another way to write above query

```
select dept_name, avg_salary
from ( select dept_name, avg (salary)
      from instructor
      group by dept_name)
      as dept_avg (dept_name, avg_salary)
where avg_salary > 42000;
```



- 例：在From子句中——用子查询构造派生关系
 - R: 选修关系
 - 问：每门课程的最高成绩和相应学生姓名？

R

姓名	课程	成绩
王红	物理	70
王红	数学	80
张军	数学	90
张军	物理	100
刘朝	物理	80

?



?

课程	最高成绩
物理	100
数学	90

课程	姓名	最高成绩
物理	张军	100
数学	张军	90



With Clause

- The **with** clause provides a way of defining a temporary relation whose definition is available only to the query in which the **with** clause occurs.
- Find all departments with the maximum budget

```
with max_budget (value) as  
    (select max(budget)  
     from department)  
select department.name  
from department, max_budget  
where department.budget = max_budget.value;
```



Complex Queries using With Clause

- Find all departments where the total salary is greater than the average of the total salary at all departments

```
with dept_total (dept_name, value) as  
    (select dept_name, sum(salary)  
     from instructor  
     group by dept_name),  
dept_total_avg(value) as  
    (select avg(value)  
     from dept_total)  
select dept_name  
from dept_total, dept_total_avg  
where dept_total.value > dept_total_avg.value;
```




Scalar Subquery

- Scalar subquery is one which is used where a single value is expected
- List all departments along with the number of instructors in each department

```
select dept_name,  
        ( select count(*)  
          from instructor  
          where department.dept_name = instructor.dept_name)  
        as num_instructors  
from department;
```

- Runtime error if subquery returns more than one result tuple



Modification of the Database

- Deletion of tuples from a given relation.
- Insertion of new tuples into a given relation
- Updating of values in some tuples in a given relation



Deletion

- Delete all instructors

delete from *instructor*

- Delete all instructors from the Finance department

delete from *instructor*

where *dept_name* = 'Finance';

- *Delete all tuples in the instructor relation for those instructors associated with a department located in the Watson building.*

delete from *instructor*

where *dept name* **in** (**select** *dept name*

from *department*

where *building* = 'Watson');



Deletion (Cont.)

- Delete all instructors whose salary is less than the average salary of instructors

```
delete from instructor  
where salary < (select avg (salary)  
                from instructor);
```

- Problem: as we delete tuples from *instructor*, the average salary changes
- Solution used in SQL:
 1. First, compute **avg** (*salary*) and find all tuples to delete
 2. Next, delete all tuples found above (without recomputing **avg** or retesting the tuples)



Delete 语句

- 例：
 - 删除不及格的学生的选修信息
 - 删除平均分不及格的学生的选修信息

R

姓名	课程	成绩
王红	物理	50
王红	数学	60
张军	数学	90
张军	化学	0
刘朝	物理	80

?



R

姓名	课程	成绩
张军	数学	90
刘朝	物理	80

?



R

姓名	课程	成绩
刘朝	物理	80



Insertion

- Add a new tuple to *course*

insert into *course*

values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

- or equivalently

insert into *course* (*course_id*, *title*, *dept_name*, *credits*)

values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

- Add a new tuple to *student* with *tot_creds* set to null

insert into *student*

values ('3003', 'Green', 'Finance', *null*);



Insertion (Cont.)

- Make each student in the Music department who has earned more than 144 credit hours an instructor in the Music department with a salary of \$18,000.

insert into *instructor*

select *ID, name, dept_name, 18000*

from *student*

where *dept_name = 'Music' and total_cred > 144;*

- The **select from where** statement is evaluated fully before any of its results are inserted into the relation.

Otherwise queries like

insert into table1 select * from table1

would cause problem



Insert 语句

- 例： 插入、从另外一个表插入
 - 1. *Create table* C (课程 varchar(20), 平均成绩 float)
 - 2. 插入两条记录： 物理 70分, 数学 85分
 - 3. 从R表中统计每门课程的平均成绩, 然后保存到C表里
 - *Insert Into* C

R

姓名	课程	成绩
王红	物理	70
王红	数学	80
张军	数学	90

C

课程	平均成绩
物理	70
数学	85

- *Group By 课程*
- *Avg(成绩)*



Updates

- Give a 5% salary raise to all instructors

update *instructor*

set *salary* = *salary* * 1.05

- Give a 5% salary raise to those instructors who earn less than 70000

update *instructor*

set *salary* = *salary* * 1.05

where *salary* < 70000;

- Give a 5% salary raise to instructors whose salary is less than average

update *instructor*

set *salary* = *salary* * 1.05

where *salary* < (**select avg** (*salary*)
from *instructor*);



Updates (Cont.)

- Increase salaries of instructors whose salary is over \$100,000 by 3%, and all others by a 5%
 - Write two **update** statements:

```
update instructor
  set salary = salary * 1.03
  where salary > 100000;
update instructor
  set salary = salary * 1.05
  where salary <= 100000;
```
 - The order is important
 - Can be done better using the **case** statement (next slide)



Case Statement for Conditional Updates

- Same query as before but with case statement

```
update instructor  
  set salary = case  
    when salary <= 100000 then salary * 1.05  
    else salary * 1.03  
  end
```



Updates with Scalar Subqueries

- Recompute and update *tot_creds* value for all students

```
update student S  
set tot_cred = (select sum(credits)  
                from takes, course  
                where takes.course_id = course.course_id and  
                    S.ID = takes.ID and  
                    takes.grade  $\neq$  'F' and  
                    takes.grade is not null);
```

- Sets *tot_creds* to null for students who have not taken any course
- Instead of **sum**(*credits*), use:

```
case  
  when sum(credits) is not null then sum(credits)  
  else 0  
end
```



Update 语句

- 例：
 - 在原有的学生关系S里面增加一个新属性：选修课程数。然后填充正确的数值。
 - 为王红插入选修课程数为2，为 张军插入选修课程数为1
 - Update S set 选修课程数 =
(? ? ?)

R

姓名	课程	成绩
王红	物理	70
王红	数学	80
张军	数学	90

count(课程)

S

姓名	选修课程数
王红	?
张军	?



End of Chapter 3